

- [NAME](#)
 - [SYNOPSIS](#)
 - [DESCRIPTION](#)
 - [COMMANDS](#)
 - [OPTIONS](#)
 - [EXIT STATUS](#)
 - [ENVIRONMENT](#)
 - [STANDARDS](#)
 - [AVAILABILITY](#)
 - [BUGS](#)
 - [SEE ALSO](#)
 - [AUTHORS](#)
-

NAME

zpaq - Journaling archiver for incremental backups.

SYNOPSIS

zpaq command archive[.zpaq] [files]... [-options]...

DESCRIPTION

zpaq manages journaling archives for incremental user-level local or remote backups that conform to The ZPAQ Open Standard Format for Highly Compressed Data (see AVAILABILITY). The format supports encrypted, deduplicated, and compressed single or multi-part archives with rollback capability. It supports archives as large as 1000 times available memory or up to 250 TB and 4 billion files, interoperable between Windows and Unix/Linux/OS X.

COMMANDS

command is one of [add](#), [extract](#), or [list](#). Commands may be abbreviated to [a](#), [x](#), or [l](#) respectively. archive is assumed to have a .zpaq extension if no extension is specified.

*If archive contains wildcards * or ?, then the archive is in multiple parts where * matches the part number and ? matches single digits. zpaq will consider the concatenation of the parts in numerical order starting with 1 to be equivalent to a single archive. For example, arc?? would match the concatenation of arc01.zpaq, arc02.zpaq, etc. up to the last existing part.*

a
add

*Append changes in files to archive, or create archive if it does not exist. files is a list of file and directory names separated by spaces. If a name is a directory, then it recursively includes all files and subdirectories within. In Windows, files may contain wildcards * and ? in the last component of the path (after the last slash). * matches any string and ? matches any character. In Unix/Linux, wildcards are expanded by the shell, which has the same effect.*

A change is an addition, update, or deletion of any file or directory in files or any of its subdirectories to any depth. A file or directory is considered changed if its size or last-modified date (with 1 second resolution), or Windows

attributes or Unix/Linux permissions (if saved) differ between the internal and external versions. File contents are not compared. If the attributes but not the date has changed, then the attributes are updated in the archive with the assumption that the file contents have not changed.

Files are added by splitting them into fragments along content-dependent boundaries, computing their SHA-1 hashes, and comparing with hashes already stored in the archive. If the hash matches, it is assumed that the fragments are identical and only a pointer to the previous compressed fragment is saved. Unmatched fragments are packed into blocks, compressed, and appended to the archive.

For each added or updated file or directory, the following information is saved in the archive: the compressed contents, fragment hashes, the file or directory name as it appears in files plus any trailing path, the last-modified date with 1 second resolution, and the Unix/Linux permissions or Windows attributes. Other metadata such as owner, group, ACLs, last access time, etc. are not saved. Symbolic links are not saved or followed. Hard links are followed as if they were ordinary files. Special file types such as devices, named pipes, and named sockets are not saved. The 64 bit Windows version will save alternate data streams.

If any file cannot be read (e.g. permission denied), then it is skipped and a warning is reported. However, other files are still added and the update is still valid.

If archive is "" (a quoted empty string), then zpaq compresses files as if creating a new archive, but discards the output without writing to disk.

If archive is multi-part, the zpaq will create a new part using the next available part number. For example:

```
zpaq add "arc??" files    (creates arc01.zpaq)
zpaq add "arc??" files    (creates arc02.zpaq)
zpaq add "arc??" files    (creates arc03.zpaq)
zpaq extract "arc??"      (extracts all parts)
```

Updates are transacted. If zpaq is interrupted before completing the update, then the partially appended data is ignored and overwritten on the next update. This is accomplished by first appending a temporary update header, appending the compressed data and index, then updating the header as the last step.

As the archive is updated, the program will report the percent complete, estimated time remaining, the name and size of the file preceded by + if the file is being added, # if updated, or - if deleted. If the file is deduplicated, then the new size after deduplication but before compression is shown.

x extract

Extract files (including the contents of directories), or extract the whole archive contents if files is omitted. The file names, last-modified date, and permissions or attributes are restored as saved in the archive. If there are multiple versions of a file stored, then only the latest version is extracted. If a stored file has been marked as deleted, then it is not extracted.

Existing files are skipped without being overwritten. (Use [-force](#) to overwrite).

As files are extracted, the fragment SHA-1 hashes are computed and compared with the stored hashes. The program reports an error in case of mismatches. Blocks are only decompressed up to the last used fragment. If the archive is damaged, then zpaq will extract as much as possible from the undamaged blocks.

As files are extracted, the program reports the percent completed, estimated time remaining, and the name of the file preceded by ">" if the file is created or overwritten (with [-force](#)), ? if the file is skipped because it already exists, or = if decompression is skipped with [-force](#) because the contents were compared and found to be identical. The date and attributes are still extracted in this case.

l list

List the archive contents. With files, list only the specified files and directories and compare them with the same files on disk. For each file or directory, show the comparison result, last modified date, uncompressed size, Windows attributes or Unix/Linux permissions, and the saved name. If the internal and external versions of the file differ, then show both.

The comparison result is reported in the first column as = if the last-modified date, attributes (if saved), and size are identical, # if different, - if the external file does not exist, or + if the internal file does not exist. With [-force](#), the contents are compared, but not the dates or attributes. Contents are compared by reading the files, computing SHA-1 hashes and comparing with the stored hashes. In either case, replacing [list](#) with [add](#) will show exactly what changes would be made to the archive.

In Unix/Linux, permissions are listed as a file type d for directory or blank for a regular file, followed by a 4 digit octal number as per `chmod(1)`. In Windows, attributes are listed from the set RHS DAdFTprCoIEivs where the character is present if the corresponding bit 0..17 is set as returned by `GetFileAttributes()`. The meanings are as follows: Read-only, Hidden, System, unused (blank), Directory, Archive, device, normal File, Temporary, sparse file, reparse point, Compressed, offline, not content indexed, Encrypted, integrity stream, virtual, no [scrub](#) data.

archive may be "", which is equivalent to comparing with an empty archive.

OPTIONS

-all [N]

With [list](#), list all saved versions and not just the latest version, including versions where the file is marked as deleted. Each version is shown in a separate numbered directory beginning with 0001/. Absolute paths are first converted to relative paths. In Windows, the : on the drive letter is removed. For example, foo and /foo are shown as 0001/foo. C:/foo and C:foo are shown as 0001/C/foo.

The date shown on the root directory of each version is the date of the update. The root directory listing also shows the number of updates and deletions in that version and the compressed size.

When a file is deleted, it is shown with the dates and attributes blank with size 0.

With [extract](#), extract the files in each version as shown with `list -all`.

N selects the number of digits in the directory name. The default is 4. More digits will be used when necessary. For example:

```
zpaq list archive -all 2 -not "??/?"
```

will show the dates when the archive was updated as 01/, 02/, etc. but not their contents.

-f -force

With [add](#), attempt to add files even if the last-modified date has not changed. Files are added only if they really are different, based on comparing the computed and stored SHA-1 hashes

With [extract](#), overwrite existing output files. If the contents differ (tested by comparing SHA-1 hashes), then the file is decompressed and extracted. If the dates or attributes/permissions differ, then they are set to match those stored in the archive.

With [list](#) files, compare files by computing SHA-1 fragment hashes and comparing with stored hashes. Ignore differences in dates and attributes.

-fragment N

Set the dedupe fragment size range from $64 \cdot 2^N$ to $8128 \cdot 2^N$ bytes with an average size of $1024 \cdot 2^N$ bytes. The default is 6 (range 4096..520192, average 65536). Smaller fragment sizes can improve compression through deduplication of similar files, but require more memory and more overhead. Each fragment adds about 28 bytes to the archive and requires about 40 bytes of memory. For the default, this is less than 0.1% of the archive size.

Values other than 6 conform to the ZPAQ specification and will decompress correctly by all versions, but do not conform to the recommendation for best deduplication. Adding identical files with different values of N will not deduplicate because the fragment boundaries will differ. `list -summary` will not identify these files as identical for the same reason.

-index indexfile

With [add](#), create archive.zpaq as a suffix to append to a remote archive which is assumed to be identical to indexfile except that indexfile contains no compressed file contents (D blocks). Then update indexfile by appending a copy of archive.zpaq without the D blocks. With [extract](#), specify the index to create for archive.zpaq and do not extract any files.

The purpose is to maintain a backup offsite without using much local disk space. The normal usage is to append the suffix at the remote site and delete it locally, keeping only the much smaller index. For example:

```
zpaq add part files -index index.zpaq
cat part.zpaq >> remote.zpaq
rm part.zpaq
```

indexfile has no default extension. However, with a .zpaq extension it can be listed to show the contents of the remote archive or compare with local files. It cannot be extracted or updated as a regular archive. Thus, the following should produce identical output:

```
zpaq list remote.zpaq
zpaq list index.zpaq
```

If archive is multi-part (contains * or ?), then zpaq will substitute a part number equal to 1 plus the number of previous updates. The parts may then be accessed as a multi-part archive without appending or renaming.

With [add](#), it is an error if the archive to be created already exists, or if indexfile is a regular archive. `-index` cannot be used with `-until` or a streaming archive `-method s...`. With [extract](#), it is an error if indexfile exists and [-force](#) is not used to overwrite.

-key password

This option is required for all commands operating on an encrypted archive. When creating a new archive with [add](#), the new archive will be encrypted with password and all subsequent operations will require the same password.

An archive is encrypted with AES-256 in CTR mode. The password is strengthened using `Script(SHA-256(password), salt, N=16384, r=8, p=1)`, which would require 208M operations and 16 MB memory per test in a brute force key search. When creating a new archive, a 32 byte salt is generated using `CryptGenRandom()` in Windows or from `/dev/urandom` in Unix/Linux, such that the first byte is different from the normal header of an unencrypted archive (z or 7). A multi-part archive is encrypted with a single keystream as if the parts were concatenated. An index is encrypted with the same password, where the first byte of the salt is modified by XOR with ('z' XOR '7').

Encryption provides secrecy but not authentication. An attacker who knows or can guess any bits of the plaintext can set them without knowing the key.

-mtype[Blocksize[.pre[.arg]][comp[.arg]]...]

-method type[Blocksize[.pre[.arg]][comp[.arg]]...]

With [add](#), select a compression method. type may be 0, 1, 2, 3, 4, 5, [x](#), or [s](#). The optional Blocksize may be 0..11, written with no space after the type, like `-m10` or `-method 511`. The remaining arguments, separated by periods or commas without spaces, are only allowed for types [x](#) or [s](#), for example `-mx4.3ci1`.

If type is numeric, then higher numbers compress better but are slower. The default is -m1. It is recommended for backups. -m2 compresses slower but decompresses just as fast as 1. It is recommended for archives to be compressed once and decompressed many times, such as downloads. -m0 stores with deduplication but no further compression.

Blocksize says to pack fragments into blocks up to $2^{\text{Blocksize}}$ MiB. Using larger blocks can improve compression but require more memory and may be slower because each block is compressed or decompressed by a separate thread. The memory requirement is up to 8 times Blocksize per thread for levels up to 4 and 16 times block size per thread for level 5. The default Blocksize is 4 (16 MiB) for types 0 and 1, and 6 (64 MiB) otherwise.

Types x and s are for experimental use. Normally, zpaq selects different methods depending on the compression level and an analysis of the data (text, executable, or other binary, and degree of compressibility). type selects journaling or streaming format. pre is 0..7 selecting a preprocessing step (LZ77, BWT, E8E9), comp is a series of context modeling components from the set {c,i,a,w,m,s,t} selecting a CM or ICM, ISSE chain, MATCH, word model, MIX, SSE, or MIX2 respectively. pre and comp may be followed by a list of numeric arguments (arg) separated by periods or commas. For example:

```
-method x6.3ci1
```

selects a journaling archive (x), block size $2^6 = 64$ MiB, BWT transform (3), an order 0 ICM (c), and order 1 ISSE (i1). (zpaq normally selects this method for level 3 text compression). type is as follows.

x

Selects normal (journaling) mode. Files are split into fragments, deduplicated, packed into blocks, and compressed by the method described. The compressed blocks are preceded by a transaction header giving the date of the update. The blocks are followed by a list of fragment hashes and sizes and a list of files added, updated, or deleted. Each added or updated file lists the last-modified date, attributes, and a list of fragment IDs.

s

Selects streaming mode for single-pass extraction and compatibility with zpaq versions prior to 6.00 (2012). Streaming archives do not support deduplication or rollback. Files are split into fragments of size $2^{\text{blocksize}}$ MiB - 4 KiB. Each file or fragment is compressed in a separate block with no attempt at deduplication. The file name, date, and attributes are stored in the header of the first fragment. The hashes are stored in the trailers of each block. There is no transaction block to allow rollback. Files are added to the previously dated update. Streaming mode with -index is an error.

pre[.min1.min2.depth.size[.lookahead]]

pre selects a pre/post processing step before context modeling as follows.

- 0 = no preprocessing
- 1 = Packed LZ77
- 2 = Byte aligned LZ77
- 3 = BWT (Burrows-Wheeler Transform)
- 4 = E8E9
- 5 = E8E9 + packed LZ77
- 6 = E8E9 + byte aligned LZ77
- 7 = E8E9 + BWT

The E8E9 transform (4..7) improves the compression of x86 executable files (.exe or .dll). The transform scans backward for 5 byte patterns of the form (E8|E9 xx xx xx 00|FF) hex and adds the block offset to the three middle bytes. The E8 and E9 opcodes are CALL and JMP, respectively. The transform replaces relative addresses with absolute addresses. The transform is applied prior to LZ77 or BWT. Decompression reverses the transforms in the opposite order.

LZ77 (1, 2, 5, 6) compresses by searching for matching strings using a hash table or suffix array and replacing them with pointers to the previous match. Types 1 and 2 select variable bit length coding or byte aligned coding respectively. Variable bit length encoding compresses better by itself, but byte aligned coding allows for further

compression using a context model. Types 6 and 7 are the same as 1 and 2 respectively, except that the block is E8E9 transformed first.

BWT (Burrows Wheeler Transform, 3 or 7), sorts the input block by context, which brings bytes with similar contexts together. It does not compress by itself, but makes the input suited to compression with a fast adapting low order context model.

The remaining arguments apply only to LZ77. *min1* selects the minimum match length, which must be at least 4 for packed LZ77 or 1 for byte aligned LZ77. *min2* selects a longer minimum match length to try first, or is 0 to skip this step. The block is encoded by testing 2^{depth} locations indexed by a hash table of 2^{size} elements indexed by hashes of the next *min2* and then *min1* characters. If *lookahead* is specified and greater than 0, then, the search is repeated *lookahead* + 1 times to consider coding the next 0 to *lookahead* bytes as literals to find a longer match.

If *size* = *blocksize* + 21, then matches are found using a suffix array instead of a hash table, scanning forward and backward 2^{depth} elements to find the longest past match. *min2* has no effect. A suffix array requires $4.5 \times 2^{\text{blocksize}}$ MiB memory. A hash table requires $4 \times 2^{\text{size}}$ bytes memory. For example:

`-method x6.1.4.0.5.27.1`

specifies 64 MiB blocks (6), variable length LZ77 without E8E9 (1), minimum match length 4, no secondary search (0), search depth $2^5 = 32$ in each direction in the suffix array ($27 = 6 + 21$), and 1 byte lookahead.

comp specifies a component of a context model. If this section is empty, then no further compression is performed. Otherwise the block is compressed by an array of components. Each component takes a context and possibly the outputs of earlier components, and outputs a prediction, a probability that the next bit of input is a 1. The final prediction is used to arithmetic code the bit. Components normally allocate memory equal to the block size, or less for smaller contexts as needed. Components are as follows:

c[.maxcount[.offset[.mask]...]

Specifies a context model (CM), or indirect context model (ICM). A CM maps a context hash to a prediction by looking up the context in a table, and then adjusts the prediction to reduce the coding error by $1/\text{count}$, where *count* is bounded by *maxcount* x 4, and *maxcount* is in 1..255.

If *maxcount* is 0, then specify an ICM. An ICM maps a context to a state representing two bit counts and the most recent bit. That state is mapped to a prediction and updated at a fixed rate. An ICM adapts faster to changing statistics. A CM with a high count compresses stationary data better. The default is 0 (ICM).

If *maxcount* has the form $1000m + n$, then the effect is the same as *maxcount* = *n* while reducing memory to $1/2^m$ of block size.

The remaining arguments represent contexts, all of which are hashed together. If *offset* is 1..255, then the block *offset mod offset* is hashed in. If *offset* is 1000..1255, then the distance to the last occurrence of *offset* - 1000 is hashed in. For example, `c0.1010` specifies an ICM taking the text column number (distance back to the last linefeed = 10) as context. The default is 0 (no context).

Each *mask* is ANDed with previous bytes. For example, `c0.0.255.255.255` is an ICM with order 3 context. A value in 256..511 specifies a context of *mask* - 256 hashed together with the byte aligned LZ77 parse state (whether a literal or match code is expected). For example, `-method x6.2.12.0.8.27c0.0.511.255` specifies block size 2^6 MiB, byte aligned LZ77 (2), minimum match length 12, search depth 2^8 , suffix array search ($27 = 6 + 21$), an ICM (c0), no offset context (0), and order 2 context plus LZ77 state (511.255).

A *mask* greater than 1000 is shorthand for *mask* - 1000 zeros. For example, the sparse context `c0.0.255.1003.255` is equivalent to `c0.0.255.0.0.0.255`.

m[size[.rate]]

Specifies a MIX (mixer). A MIX computes a weighted average of the predictions of all previous components. (The averaging is in the logistic domain: $\log(p / (1 - p))$). The weights are then adjusted in proportion to *rate*

(0..255) to reduce the prediction error. A size bit context can be used to select a set of weights to be used. The first 8 bits of context are the previously coded bits of the current byte. The default is `m8.24`. A MIX with n inputs requires $4n \times 2^{\text{size}}$ bytes of memory.

`t[size[.rate]]`

Specifies a MIX2. A MIX2 is like a MIX except that it takes only the last 2 components as input, and its weights are constrained to add to 1. A MIX2 requires $4 \times 2^{\text{size}}$ bytes of memory. The default is `t8.24`.

`s[size[.mincount[.maxcount]]]`

Specifies a SSE (secondary symbol estimator). A SSE takes the last size bits of context and the quantized and interpolated prediction of the previous component as input to output an adjusted prediction. The output is adjusted to reduce the prediction error by $1/\text{count}$, where the count is constrained between mincount and $4 \times \text{maxcount}$. The default is `s8.32.255`.

`iorder[.increment]...`

Specifies an ISSE (indirect secondary symbol estimator) chain. An ISSE adjusts the prediction of the previous component by mixing it with a constant 1. The pair of mixing weights is selected by a bit history state (like an ICM). The bit history is selected by a hash of the last order bytes hashed together with the context of the previous component. Each increment specifies an additional ISSE whose context order is increased by increment. For example, `ci1.1.2` specifies an order 0 ICM and order 1, 2, and 4 ISSEs.

`w[order[.A[.Z[.cap[.mul[.mem]]]]]]`

Specifies an ICM-ISSE chain of length order taking as contexts the hashes of the last 1, 2, 3..., order whole words. A word is defined as a sequence of characters in the range A to A + Z - 1, ANDed with cap before hashing. The hash H is updated by byte c as $H := (H \times \text{mul} + c) \pmod{2^{(\text{blocksize} + 24 - \text{mem})}}$. Each component requires $2^{(\text{blocksize} - \text{mem})}$ MiB. The default is `w1.65.26.223.20.0`, which defines a word as 65..90 (A..Z). ANDing with 223 converts to upper case before hashing. `mul = 20` has the effect of shifting 2 bits left. For typical block sizes (28 or 30 bit H), the word hash depends on the last 14 or 15 letters.

`a[mul[.bmem][.hmem]]`

Specifies a MATCH. A MATCH searches for a past matching context and predicts whatever bit came next. The search is done by updating a context hash H with byte c by $H := H \times \text{mul} + c \pmod{2^{(\text{blocksize} + 18 - \text{hmem})}}$. A MATCH uses $2^{(\text{blocksize} - \text{bmem})}$ MiB history buffer and a $2^{(\text{blocksize} - \text{hmem})}$ MiB hash table. The default is `a24.0.0`. If blocksize is 6, then H is 24 bits. `mul = 24` shifts 4 bits left, making the context hash effectively order 6.

`-noattributes`

With [add](#), do not save Windows attributes or Unix/Linux permissions to the archive. With [extract](#), ignore the saved values and extract using default values. With [list](#), do not list or compare attributes.

`-not [file]...`

`-not =[#+-?^]...`

In the first form, do not add, extract, or list files that match any file by name. file may contain wildcards * and ? that match any string or character respectively, including /. A match to a directory also matches all of its contents. In Windows, matches are not case sensitive, and \ matches /. In Unix/Linux, arguments with wildcards must be quoted to protect them from the shell.

When comparing with [list](#) files, [-not =](#) means do not list identical files. Additionally it is possible to suppress listing of differences with #, missing external files with -, missing internal files with +, and duplicates (`list -summary`) with ^.

`-only file...`

Do not add, extract, or list any files unless they match at least one argument. The rules for matching wildcards are the same as [-not](#). The default is `*` which matches everything.

If a file matches an argument to both `-only` and [-not](#), then [-not](#) takes precedence.

`-repack new_archive [new_password]`

With [extract](#), store the extracted files in `new_archive` instead of writing them individually to disk. If `new_password` is specified, then the output is encrypted with this password. Otherwise the output is not encrypted, even if the input is.

It is an error if `new_archive` exists unless [-force](#) is used to allow it to be overwritten. `new_archive` does not automatically get a `.zpaq` extension.

Repacking is implemented by copying those D blocks (compressed file contents) which are referenced by at least one selected file. This can result in a larger archive than a new one because unreferenced fragments in the same block are also copied.

The repacked archive block dates range from the first to last update of the input archive. Using `add -until` with a date between these two dates will result in the date being adjust to 1 second after the last update.

With `-all`, the input archive is simply copied without modification except to decrypt and encrypt. Thus, the input may be any file, not just an archive. files and the options `-to`, [-not](#), `-only`, `-until`, [-noattributes](#), and `-method` are not valid with `-repack -all`.

`-sN`

`-summary N`

With [list](#), sort by decreasing size and show only the `N` largest files and directories. Label duplicates of the previous file with `^`. A file is a duplicate if its contents are identical (based on stored hashes) although the name, dates, and attributes may differ. If `files` is specified, then these are included in the listing but not compared with internal files or each other. Internal and external files are labeled with `-` and `+` respectively.

If `N` is negative as in `-s-1` then list normally but show fragment IDs after each file name. Files with identical fragment IDs have identical contents.

With [add](#) and [extract](#), when `N > 0`, do not list files as they are added or extracted. Show only percent completed and estimated time remaining on a 1 line display.

`-test`

With [extract](#), do not write to disk, but perform all other operations normally. [extract](#) will decompress, compute the SHA-1 hashes of the output, report if it differs from the stored value, but not compare, create or update any files. With `-index`, test for errors but do not create an index file.

`-tN`

`-threads N`

Add or extract at most `N` blocks in parallel. The default is 0, which uses the number of processor cores, except not more than 2 when `zpaq` is compiled to 32-bit code. Selecting fewer threads will reduce memory usage but run slower. Selecting more threads than cores does not help.

`-to name...`

With [add](#) and [list](#) rename external files to respective internal names. With [extract](#), rename internal files to external names. When `files` is empty, prefix the extracted files with the first name in `names`, inserting `/` if needed and removing `:` from drive letters. For example:

```
zpaq extract archive file dir -to newfile newdir
```

extracts file as newfile and dir as newdir.

```
zpaq extract archive -to tmp
```

will extract foo or /foo as tmp/foo and extract C:/foo or C:foo as tmp/C/foo.

```
zpaq add archive dir -to newdir
```

will save dir/file as newdir/file, and so on.

```
zpaq list archive dir -to newdir
```

will compare external dir with internal newdir.

The -only and [-not](#) options apply prior to renaming.

-until date | [-]version

Ignore any part of the archive updated after date or after version updates or -versions from the end if negative. Additionally, [add](#) will truncate the archive at this point before appending the next update. When a date is specified, the update will be timestamped with date rather than the current date.

A date is specified as a 4 digit year (1900 to 2999), 2 digit month (01 to 12), 2 digit day (01 to 31), optional 2 digit hour (00 to 23, default 23), optional 2 digit minute (00 to 59, default 59), and optional 2 digit seconds (00 to 59, default 59). Dates and times are always universal time zone (UT), not local time. Numbers up to 9999999 are interpreted as version numbers rather than dates. Dates may contain spaces and punctuation characters for readability but are ignored. For example:

```
zpaq list backup -until 3
```

shows the archive as it existed after the first 3 updates.

```
zpaq add backup files -until 2014/04/30 11:30
```

truncates any data added after April 30, 2014 at 11:30:59 universal time, then appends the update as if this were the current time. (It does not matter if any files are dated in the future).

```
zpaq add backup files -until 0
```

deletes backup.zpaq and creates a new archive.

add -until is an error on multi-part archives or with an index. A multi-part archive can be rolled back by deleting the highest numbered parts.

Truncating and appending an encrypted archive with add -until (even -until 0) does not change the salt or keystream. Thus, it is possible for an attacker with the old and new versions to obtain the XOR of the trailing plaintexts without a password.

EXIT STATUS

Returns 0 if successful, 1 in case of warnings, or 2 in case of an error.

ENVIRONMENT

In Windows, the default number of threads (set by -threads) is %NUMBER_OF_PROCESSORS%. In Linux, the number of lines of the form "Processor : 0", "Processor : 1",... in /cpu/procinfo is used instead.

STANDARDS

The archive format is described in The ZPAQ Open Standard Format for Highly Compressed Data (see AVAILABILITY).

AVAILABILITY

<http://mattmahoney.net/zpaq/>

BUGS

There is no GUI.

The archive format does not save sufficient information for backing up and restoring the operating system.

SEE ALSO

bzip2(1) gzip(1) lrzip(1) lzop(1) lzma(1) p7zip(1) rzip(1) unace(1) unrar(1) unzip(1) zip(1)

AUTHORS

zpaq and libzpaq are written by Matt Mahoney and released to the public domain in 2015-2016. libzpaq contains libdivsufsort-lite v2.01, copyright (C) 2003-2008, Yuta Mori. It is licensed under the MIT license. See the source code for license text. The AES code is modified from libtomcrypt by Tom St Denis (public domain). The salsa20/8 code in Scrypt() is by D. J. Bernstein (public domain).